ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №5

«Обробка помилок, файли та серіалізація»

з курсу «Професійний Python»
Варіант №13 — «Система обліку комп'ютерної техніки»

Виконав: Капарник Дмитро Андрійович

Група: ФЕП-32

Перевірив: доцент Куньо Іван Михайлович

Дата: 18.10.2026

Львів — 2026

1. Мета роботи

Розробити надійний імпорт та експорт з конфігурацією, обробленням помилок, журналюванням і контрольованим використанням файлів.

2. Індивідуальний варіант і постановка задачі

Варіант №13 — система обліку комп'ютерної техніки. Перевіряти inventory number, type/manufacturer/model та value >= 0; YAML задає дозволені типи й виробників. Реалізувати CSV import, JSON export, окремий error log, статистику та atomic output.

Наскрізний проєкт продовжує ЛР №1. Базові поля: інвентарний номер, тип, виробник, модель і вартість. Збережено пошук, фільтр виробника, середню/загальну вартість і визначення найдорожчого пристрою.

3. Структура проєкту та створені модулі

```
inventory-service/
  pyproject.toml, README.md, .gitignore
  src/computer_inventory/
    models.py
    fileio.py
    main.py
  src/inventory_service/  # фінальний пакет запуску
  tests/
  scripts/
  docs/GITHUB.md
```

4. Схема даних і конфігурація

```
inventory_number,type,manufacturer,model,value,status
INV-001,Computer,Dell,Latitude,45000.00,active
INV-002,Printer,HP,LaserJet,7200.00,maintenance
INV-003,Computer,Lenovo,ThinkPad,32000.00,active
```

CSV ЛР5 використовує value; це та сама вартість, що cost у моделі ЛР1 та CSV ЛР3. Імпортер підтримує обидві назви. Status — додаткове поле політики імпорту: active, maintenance або retired; воно перевіряється, але в базовий Device не додається. Значення вартості в JSON записується рядком для збереження копійок.

```
input: data/inventory.csv
output: data/export.json
allowed_types: [Computer, Printer]
allowed_manufacturers: [Dell, HP, Lenovo]
allowed_statuses: [active, maintenance, retired]
skip_invalid: false
```

```
@dataclass(frozen=True)
class FileConfig:
```

```
    input: Path
    output: Path
    allowed_types: tuple[str, ...] = ()
    allowed_manufacturers: tuple[str, ...] = ()
    allowed_statuses: tuple[str, ...] = ("active", "maintenance", "retired")
    skip_invalid: bool = False

def load_config(path: Path) -> FileConfig:
    try:
        with path.open(encoding="utf-8") as source:
            data = yaml.safe_load(source)
        if not isinstance(data, dict):
            raise ValueError("Configuration must be a mapping")
        for key in ("input", "output"):
            if not isinstance(data.get(key), str) or not data[key].strip():
                raise ValueError(f"Missing or invalid {key}")
        for key in ("allowed_types", "allowed_manufacturers", "allowed_statuses"):
            values = data.get(key, [])
            if not isinstance(values, list) or any(
                not isinstance(x, str) or not x.strip() for x in values
            ):
                raise ValueError(f"Invalid {key}")
        skip = data.get("skip_invalid", False)
        if not isinstance(skip, bool):
            raise ValueError("skip_invalid must be boolean")
        base = path.resolve().parent
        result = FileConfig(
            base / data["input"],
            base / data["output"],
            tuple(data.get("allowed_types", [])),
            tuple(data.get("allowed_manufacturers", [])),
            tuple(data.get("allowed_statuses", ["active", "maintenance", "retired"])),
            skip,
        )
        if not result.input.is_file():
            raise ValueError("Input file does not exist")
        return result
    except (OSError, yaml.YAMLError, ValueError) as error:
        raise ConfigurationError(f"Cannot load configuration: {path}") from error
```

Відносні шляхи визначаються від каталогу YAML. Перевіряються обов’язкові ключі, типи списків, булевий skip_invalid та наявність input. Відсутній каталог output створюється до експорту. Невідомі або заборонені дані відхиляються згідно з політикою.

5. Помилки, журналювання та потоковий імпорт

```
class InventoryError(Exception):
    """Base application error."""

class ConfigurationError(InventoryError):
    """Invalid or inaccessible YAML configuration."""

class RecordValidationError(InventoryError):
    """Invalid equipment record."""
```

```
class ImportExportError(InventoryError):
    """Input or output operation failed."""

def configure_logging(path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    handler = RotatingFileHandler(
        path, maxBytes=1000000, backupCount=2, encoding="utf-8"
    )
    handler.setFormatter(logging.Formatter("%(levelname)s %(message)s"))
    logger.addHandler(handler)
    logger.setLevel(logging.INFO)

def validate_record(row: dict[str, str], config: FileConfig) -> Device:
    try:
        if None in row or any(v is None for v in row.values()):
            raise ValueError("Malformed CSV record")
        cost = row.get("value", row.get("cost", ""))
        device = Device(
            row["inventory_number"],
            row["type"],
            row["manufacturer"],
            row["model"],
            Decimal(cost),
        )
        if config.allowed_types and device.device_type not in config.allowed_types:
            raise ValueError("Device type is not allowed")
        if (
            config.allowed_manufacturers
            and device.manufacturer not in config.allowed_manufacturers
        ):
            raise ValueError("Manufacturer is not allowed")
        status = row.get("status", "active")
        if status not in config.allowed_statuses:
            raise ValueError("Status is not allowed")
        return device
    except (KeyError, ValueError, InvalidOperation) as error:
        raise RecordValidationError("Invalid inventory record") from error

def import_csv(config: FileConfig, stats: ImportStats) -> Iterator[Device]:
    seen: set[str] = set()
    try:
        with config.input.open(encoding="utf-8-sig", newline="") as source:
            reader = csv.DictReader(source)
            required = {"inventory_number", "type", "manufacturer", "model"}
            if not required <= set(reader.fieldnames or []) or not (
                {"cost", "value"} & set(reader.fieldnames or [])
            ):
                raise ImportExportError("Required CSV columns are missing")
            for line, row in enumerate(reader, 2):
                stats.processed += 1
                try:
                    device = validate_record(row, config)
                    key = device.inventory_number.casefold()
                    if key in seen:
                        raise RecordValidationError("Duplicate inventory number")
                    seen.add(key)
```

```
                except RecordValidationError:
                    stats.invalid += 1
                    logger.exception("Invalid record at line %d", line)
                    if not config.skip_invalid:
                        raise
                    continue
                stats.valid += 1
                yield device
    except (OSError, csv.Error) as error:
        raise ImportExportError("CSV input failed") from error
```

Exception hierarchy відокремлює конфігураційні помилки, помилки записів та I/O. raise ... from error зберігає причину, наприклад Decimal.InvalidOperation → RecordValidationError. RotatingFileHandler записує помилки й traceback у UTF-8 та обмежує розмір журналу. Strict зупиняє операцію на першій помилці; tolerant пропускає її та збільшує invalid.

6. Серіалізація та атомарний вихід

```
@contextmanager
def atomic_text(path: Path) -> Iterator[TextIO]:
    path.parent.mkdir(parents=True, exist_ok=True)
    fd, name = tempfile.mkstemp(prefix=path.name + ".", suffix=".tmp", dir=path.parent)
    temporary = Path(name)
    try:
        with os.fdopen(fd, "w", encoding="utf-8", newline="") as target:
            yield target
            target.flush()
            os.fsync(target.fileno())
        os.replace(temporary, path)
    finally:
        temporary.unlink(missing_ok=True)

def device_json(device: Device) -> dict[str, str]:
    return {
        "inventory_number": device.inventory_number,
        "type": device.device_type,
        "manufacturer": device.manufacturer,
        "model": device.model,
        "value": f"{device.cost:.2f}",
    }

def export_json(devices: Iterator[Device], path: Path, stats: ImportStats) -> None:
    exported = 0
    try:
        with atomic_text(path) as target:
            target.write("[\n")
            for device in devices:
                if exported:
                    target.write(",\n")
                json.dump(device_json(device), target, ensure_ascii=False)
                exported += 1
            target.write("\n]\n")
        stats.exported = exported
        logger.info("Exported %d devices", exported)
```

```
    except OSError as error:
        logger.exception("Output failed")
        raise ImportExportError("JSON output failed") from error
```

Власний context manager готує temporary file у тому самому каталозі, записує, flush/fsync та публікує через os.replace. При винятку тимчасовий файл прибирається, а попередній output зберігається. Статистика exported присвоюється після успішної заміни. Атомарна заміна забезпечує цілісний файл у межах файлової системи; це не універсальна гарантія відмовостійкості носія.

7. Експеримент strict/tolerant та аварійний експорт

Режим	Оброблено	Valid	Invalid	Exported	Завершення
strict	101	100	1	0	RecordValidationError caused by InvalidOperation
tolerant	1010	1000	10	1000	success

У вхідному файлі 1000 коректних і 10 некоректних записів. Перший некоректний стоїть після 100 valid, тому strict завершується на 101-му записі. У strict попередній JSON лишився незмінним; tolerant експортував рівно 1000 пристроїв. Окрема імітація RuntimeError після часткового запису теж зберегла original і не залишила .tmp.

```
ERROR Invalid record at line 102
Traceback (most recent call last):
  File "C:\Users\user\Documents\Codex\2026-10-18\new-chat-2\outputs\computer-
inventory\src\computer_inventory\fileio.py", line 102, in validate_record
    row["model"], Decimal(cost))
                  ^^^^^^^^^^^^
decimal.InvalidOperation: [<class 'decimal.ConversionSyntax'>]

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "C:\Users\user\Documents\Codex\2026-10-18\new-chat-2\outputs\computer-
inventory\src\computer_inventory\fileio.py", line 126, in import_csv
    device = validate_record(row, config)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\user\Documents\Codex\2026-10-18\new-chat-2\outputs\computer-
inventory\src\computer_inventory\fileio.py", line 112, in validate_record
    raise RecordValidationError("Invalid inventory record") from error
computer_inventory.fileio.RecordValidationError: Invalid inventory record
ERROR Invalid record at line 102
Traceback (most recent call last):
```

8. Експеримент streaming/eager та конфігураційні помилки

Записів	Режим	Час, с	Peak, МіБ
10000	eager	0.2034	4.952

Записів	Режим	Час, с	Peak, МіБ
10000	streaming	0.2028	1.032
100000	eager	2.2087	48.101
100000	streaming	2.0231	9.936
500000	eager	12.2483	236.498
500000	streaming	12.6314	40.831

Обидва варіанти виконують однакову валідацію й мають однакові суми. Імпортер зберігає seen для глобальної унікальності номерів, тому його streaming-пам'ять зростає з n; вона все ж значно менша за зберігання всіх Device. Ця умова відрізняє результат від майже сталого стану конвеєра ЛР3.

Перевірка	Очікувана поведінка	Підтвердження
Відсутній YAML / некоректний YAML	ConfigurationError з причиною	pytest test_missing_configuration / test_invalid_configuration
Немає input/output або хибний тип	ConfigurationError	Параметризовані тести
Відсутній input file	ConfigurationError або ImportExportError	Тести конфігурації та I/O
Тип, виробник, status не дозволені	RecordValidationError	test_record_policy
Каталог output відсутній	Створення батьківського каталогу	atomic_text та integration tests
Помилка заміни файла	ImportExportError, очищення .tmp	test_export_os_failure

```
python -m inventory_service.main --import-config config.yaml
```

Критична помилка імпорту в main завершує процес із кодом 1 та зрозумілим повідомленням. Успішний виклик повертає ImportStats. Результат запуску експерименту збережений у results05.json.

Код і історія лабораторних: https://github.com/dimakaparnyk/professional-python-labs. Для відповідного етапу використовується тег lab-05, фінальна інтегрована реалізація — main (перевірений CI commit 2e3fb3d).

9. Відповіді на контрольні запитання

1. Що таке exception?

Об'єкт, що описує помилкову або виняткову ситуацію і змінює потік виконання.

2. Для чого використовується try?

Виділяє блок коду, для якого визначено оброблення винятків.

3. Для чого except?

Перехоплює винятки зазначених типів і виконує відповідний обробник.

4. Коли виконується else після try?

Коли блок try завершився без винятку.

5. Коли виконується finally?

Під час виходу з конструкції незалежно від успіху, помилки чи return, окрім аварійної зупинки процесу.

6. Для чого використовується raise?

Для підняття винятку та передавання проблеми викликачу.

7. Що означає raise без аргументів?

Повторно піднімає поточний оброблюваний виняток зі збереженням traceback.

8. Чому не рекомендується except: pass?

Він приховує помилки й може залишити систему в некоректному стані.

9. Чому потрібно перехоплювати конкретні exception types?

Щоб не приховувати неочікувані дефекти та коректно реагувати на конкретні причини.

10. Що таке exception hierarchy?

Успадкована система типів винятків від загальних до спеціалізованих.

11. Для чого створювати custom exceptions?

Щоб виразити помилки предметної області та відокремити їх від деталей бібліотек.

12. Що таке exception chaining?

Збереження причинного зв’язку між початковим і перетвореним винятками.

13. Що означає raise ... from ...?

Піднімає новий виняток і явно задає попередній як його __cause__.

14. Яку перевагу дає chaining для debugging?

Дає змогу бачити і зрозумілий контекст застосунку, і початкову технічну причину.

15. Що означає EAFP?

Easier to Ask Forgiveness than Permission: спробувати операцію та обробити очікувану помилку.

16. Що означає LBYL?

Look Before You Leap: перевірити передумови перед операцією.

17. Що таке fail-fast?

Раннє завершення при порушенні критичних передумов замість продовження з некоректними даними.

18. Де доцільно застосовувати fail-fast?

Під час завантаження конфігурації, перевірки схеми та критичних параметрів запуску.

19. Що таке context manager?

Об’єкт, який керує входом у контекст і гарантованим завершенням роботи з ресурсом.

20. Для чого використовується with?

Для обмеження часу життя ресурсу та виконання його очищення.

21. Які methods утворюють context manager protocol?

__enter__() та __exit__(); для асинхронного контексту — __aenter__() та __aexit__().

22. Що робить __enter__()?

Готує ресурс і повертає значення для конструкції as.

23. Що робить __exit__()?

Звільняє ресурс, отримує інформацію про виняток і може вирішити, чи пригнічувати його.

24. Чи закриває with файл при exception?

Так, для стандартного файлового контексту файл закривається і при винятку.

25. Для чого contextlib.contextmanager?

Для створення контекстного менеджера з генераторної функції.

26. Яку роль має yield у generator-based context manager?

Розділяє підготовку ресурсу та завершення; функція повинна видати значення рівно один раз.

27. Що таке pathlib.Path?

Об’єктне представлення файлового шляху зі зручними операціями читання, запису та комбінування.

28. Чому pathlib кращий за ручну конкатенацію шляхів?

Враховує роздільники платформ і зменшує ризик помилок під час побудови шляху.

29. Для чого задається encoding="utf-8"?

Щоб однаково декодувати український текст незалежно від системної локалі.

30. Що таке CSV?

Текстовий табличний формат із роздільниками полів і правилами екранування.

31. Для чого csv.reader?

Для послідовного читання CSV-рядків як списків полів.

32. Для чого csv.DictReader?

Для читання рядків як словників за назвами колонок.

33. Чим DictReader зручний для structured data?

Дозволяє звертатися до полів за назвами замість залежності від їхніх позицій.

34. Для чого csv.DictWriter?

Для запису словників у CSV з визначеним порядком колонок.

35. Що таке JSON?

Текстовий формат обміну структурованими значеннями: об’єктами, масивами, рядками й числами.

36. Що таке serialization?

Перетворення об’єктів у представлення для збереження або передавання.

37. Що таке deserialization?

Відновлення структурованих значень із серіалізованого представлення.

38. Чим json.dump() відрізняється від json.dumps()?

json.dump записує у файловий об’єкт, json.dumps повертає рядок.

39. Чим json.load() відрізняється від json.loads()?

json.load читає з файлового об’єкта, json.loads розбирає переданий рядок або байти.

40. Що таке JSONDecodeError?

Виняток синтаксичного розбору некоректного JSON.

41. Для чого використовується YAML?

Для зручної для людини конфігурації та інших структурованих текстових даних.

42. Чому рекомендується yaml.safe_load()?

Він не конструює довільні Python-об’єкти з небезпечних YAML-тегів, але дані все одно треба перевіряти.

43. Які типові сценарії використання JSON?

API, конфігураційні повідомлення, експорт і сумісний обмін між мовами.

44. Які типові сценарії використання YAML?

Конфігурація застосунків, інфраструктурні файли та параметри конвеєра.

45. Що таке configuration file?

Файл із параметрами запуску та політиками, відокремленими від логіки програми.

46. Чому configuration краще не hard-code у Python?

Щоб змінювати середовище й правила без редагування та перевидання програмного коду.

47. Що потрібно перевіряти у configuration?

Обов’язкові ключі, типи, дозволені значення, діапазони, шляхи та взаємну узгодженість параметрів.

48. Що таке logging?

Керований запис подій застосунку з рівнями важливості й контекстом.

49. Чим logging відрізняється від print()?

Підтримує рівні, формати, різні обробники, файли й ротацію; print переважно виводить текст у stdout.

50. Які існують logging levels?

DEBUG, INFO, WARNING, ERROR, CRITICAL.

51. Для чого logger.info()?

Для повідомлень про нормальні важливі події, наприклад успішний експорт.

52. Для чого logger.warning()?

Для проблем, після яких програма може коректно продовжити роботу.

53. Для чого logger.error()?

Для помилок операції без обов’язкового додавання traceback.

54. Для чого logger.exception()?

Для запису повідомлення з traceback поточного винятку всередині except.

55. Що містить traceback?

Ланцюжок викликів і місця виникнення та поширення помилки.

56. Чому secrets не можна записувати в logs?

Журнали часто копіюються й доступні ширшому колу осіб; запис секрету створює витік.

57. Що таке streaming I/O?

Читання або запис частинами, які обробляються послідовно.

58. Яка перевага streaming для великих files?

Не потрібно одночасно утримувати весь набір об’єктів; додаткові індекси можуть усе ж зростати з n.

59. Що таке eager file loading?

Повне завантаження даних перед подальшою обробкою.

60. Чому list(reader) може бути небажаним?

Матеріалізує всі записи й може потребувати значного обсягу пам’яті.

61. Що таке strict validation mode?

Режим, у якому перший некоректний запис перериває операцію.

62. Що таке tolerant mode?

Режим, у якому некоректні записи журналюються й пропускаються зі статистикою помилок.

63. Коли доцільно використовувати strict mode?

Коли частковий результат неприпустимий, наприклад для узгодженого реєстру чи критичного імпорту.

64. Коли tolerant mode?

Для масових неідеальних даних, коли дозволено отримати результат із коректних записів.

65. Що таке atomic write?

Публікація готового файла однією операцією заміни після успішного завершення запису.

66. Для чого використовується temporary file?

Щоб готувати новий результат окремо й не пошкоджувати чинний файл до успіху.

67. Що може статися при звичайному output write та аварійному завершенні?

Файл може бути перезаписаний лише частково й втратити попередній коректний вміст.

68. Навіщо видаляти temporary file після error?

Щоб не накопичувати сміття та не залишати часткові результати, які можна помилково використати.

69. Що таке JSON Lines?

Формат, у якому кожний рядок є окремим JSON-значенням.

70. Чому JSON Lines зручний для streaming?

Дозволяє читати й записувати один запис за раз без матеріалізації всього JSON-масиву.

10. Висновок

Реалізовано керований YAML імпорт техніки, валідацію, статистику та атомарний JSON-експорт. Експерименти підтвердили різницю strict/tolerant і збереження попереднього результату після помилки. Потокова обробка знижує пікову пам’ять, але індекс унікальності потребує O(n).